



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 6

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 09/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
3. Desarrollo	5
3.1. Ejercicio 1.	5
3.2. Ejercicio 2.	7
4. Resultados	8
4.1. Ejercicio 1.	8
4.2. Resultados del Ejercicio 2.	9
5. Conclusiones	14

1. Introducción

1.1. Planteamiento del problema

El problema consiste en investigar mediante nuestro LLM las mejores prácticas para el uso de archivos JAR y la correcta generación de documentación JavaDoc, para posteriormente aplicar estos conocimientos en la creación y configuración de un paquete general estructurado (bajo la ruta `mx.unam.fi.die.poo.g7`) que organice todo el código, prácticas y proyectos desarrollados en el curso hasta el momento.

1.2. Motivación

Dar solución a este problema permite organizar jerárquicamente los proyectos del curso para evitar conflictos de nombres, empaquetarlos para su distribución mediante archivos JAR y garantizar su mantenimiento a largo plazo generando documentación estandarizada con JavaDoc.

1.3. Objetivos

- **General:** Organizar adecuadamente las clases según su funcionalidad o propósito bajo un namespace o paquete.
- **Específicos:**
 - Preguntarle a nuestro LLM las mejores prácticas de la industria para el uso y creación de archivos JAR y JavaDoc, el método y las herramientas adecuadas para la generación de la documentación técnica automatizada (JavaDoc).
 - Configurar el paquete personalizado `mx.unam.fi.die.poo.g7` para organizar jerárquicamente las clases desarrolladas hasta el momento y evitar conflictos de nombres con otras bibliotecas.
 - Crear una estructura de directorios (`src`, `lib`, `test`, `doc`) que separe lógicamente el código fuente (source code), las bibliotecas (libraries) de terceros, las pruebas (test) y la documentación (doc).

2. Marco Teórico

1. **Package (empaquetamiento):** Es una agrupación de clases para acomodar las clases de la biblioteca en grupos que permanecen juntos. La API está organizada en paquetes con la intención de organizar los contenidos del lenguaje. [3]
2. **El nombre completo o nombre calificado (Fully Qualified Name):** Es un identificador único de clases y subclases hasta llegar a la clase principal. [3]
3. **Jerarquías de paquetes:** Las clases del API están organizadas esta organización en jerarquías ayuda a encontrar clases particulares. [2]
4. **API de java:** Proporciona una interfaz Java con un conjunto de clases de programa de utilidad y métodos que puede utilizar para escribir código Java. [1]

3. Desarrollo

3.1. Ejercicio 1.

Este ejercicio consistió en colaborar con nuestro LLM preguntándole:

1. Buenas prácticas para uso de JAR y JavaDoc.
2. ¿Cuál es la mejor manera de generar un JavaDoc?

A lo que nos respondió lo siguiente en el caso de las buenas prácticas:

- **Automatiza la construcción:** Nunca generes archivos JAR manualmente desde la terminal. Utiliza herramientas de construcción como Maven o Gradle para asegurar la reproducibilidad, la correcta inclusión de recursos y la gestión de dependencias.
- **Versionado estandarizado:** El nombre del archivo JAR debe incluir siempre el identificador del artefacto y su versión semántica (ej. motor-pagos-2.1.0.jar).
- **Configura correctamente el MANIFEST.MF:** Aprovecha el archivo META-INF/MANIFEST.MF para incluir metadatos esenciales. Asegúrate de incluir propiedades como Implementation-Title, Implementation-Version y Build-Jdk. Si es un ejecutable, la propiedad Main-Class es obligatoria.

Separa Librerías de Aplicaciones (Fat JARs vs. Thin JARs):

- **Librerías (Thin JAR):** Si estás empaquetando una librería para que otros la usen, el JAR solo debe contener tu código compilado. No incluyas dependencias transitivas dentro de él, ya que esto causará conflictos de versiones (dependency hell) en el proyecto que lo consuma.
- **Aplicaciones (Fat/Uber JAR):** Si es un microservicio o aplicación ejecutable, utiliza plugins (como Maven Shade o Spring Boot Maven Plugin) para crear un "Fat JAR" que empaquete tu código junto con todas sus dependencias. Esto permite ejecutar la aplicación fácilmente con `java -jar app.jar`.
- **Modularidad (Java 9+):** Si tu proyecto utiliza el Sistema de Módulos de Java (JPMS), incluye tu archivo `module-info.class` en la raíz del JAR para definir explícitamente qué paquetes exporta tu librería y qué módulos requiere para funcionar.
- **Firma de seguridad:** Si vas a distribuir el JAR en entornos públicos o corporativos estrictos, firma el archivo utilizando `jarsigner` para garantizar que el código no ha sido alterado desde su empaquetado.

Buenas Prácticas para JavaDoc

- **Explica el "Qué" el "Por qué", no el "Cómo":** La documentación debe describir la intención del método, su contrato y sus reglas de negocio. El código fuente ya explica los detalles de implementación. Si cambias el código interno, el JavaDoc idealmente no debería necesitar reescribirse.

- **La primera oración es clave:** La herramienta Javadoc utiliza la primera oración (hasta el primer punto) como resumen en la vista de índices. Haz que esta primera línea sea concisa y directa al grano.
- **Evita el ruido y la redundancia:** No documentes lo que es obvio. Un método `getNombre()` no necesita un Javadoc que diga ".ºbtiene el nombre". Resérvalo para aclarar reglas específicas (ej. Retorna el nombre en formato CamelCase. Retorna nulo si el usuario es invitado").

Usa las etiquetas estándar rigurosamente:

- **@param:** Describe restricciones clave (rango de valores válidos, si acepta o no null, unidades de medida si es un número).
- **@return:** Explica exactamente qué estado o valor se devuelve. Si devuelve una colección, especifica si puede estar vacía o si alguna vez retorna null.
- **@throws / @exception:** Documenta todas las excepciones comprobadas (checked) y las no comprobadas (unchecked) que sean críticas para el negocio, explicando las condiciones exactas bajo las cuales ocurren.
- **Aprovecha el formato HTML y los enlaces:** Estructura las descripciones largas usando etiquetas HTML (`<p>`, `<ul>`, `<code>`). Usa `@link ClaseMetodo()` para crear hipervínculos a otras partes de tu API, lo que facilita la navegación del desarrollador que consume tu código.
- **Documenta a nivel de Clase/Interfaz:** Cada clase pública debe tener un bloque Javadoc que explique su responsabilidad general, su papel en la arquitectura y ejemplos básicos de uso. Incluye `@since` para indicar en qué versión se introdujo.

En el caso de ¿Cuál es la mejor manera de generar un Javadoc? nos respondió:

1. La forma estándar y recomendada: Build Tools Integrar Javadoc en tu ciclo de construcción asegura que siempre refleje la última versión de tu código fuente y maneja automáticamente el classpath de tus dependencias.

Con Maven: Se utiliza el `maven-javadoc-plugin`. Solo necesitas ejecutar en tu terminal:
Bash `mvn javadoc:javadoc`

2. Para desarrollo local: Tu IDE (IntelliJ IDEA o Eclipse) Si estás escribiendo código y quieres ver rápidamente cómo luce tu documentación (o exportarla rápido sin usar Maven/Gradle), los IDEs tienen herramientas visuales excelentes:

- **IntelliJ IDEA:** Ve al menú superior y selecciona `Tools > Generate Javadoc....` Te permitirá elegir si quieres documentar todo el proyecto o un paquete específico, y ajustar el nivel de visibilidad (Public, Protected, Private).
- **Eclipse:** Ve al menú superior `Project > Generate Javadoc....` Te abrirá un asistente para configurar la salida.

3. El nivel profesional: Integración Continua (CI/CD) y Hosting La mejor manera de distribuir un JavaDoc generado es no obligar a otros desarrolladores a generarlo en sus máquinas. Lo ideal es que configures un pipeline (por ejemplo, en GitHub Actions, GitLab CI o Jenkins) que siga estos pasos:
 - Detecta cuando un nuevo código hace "merge" a la rama main o se crea un nuevo Release/Tag".
 - Ejecuta `mvn javadoc:javadoc`.
 - Publica automáticamente la carpeta generada en un servidor estático. Muchas librerías open source usan GitHub Pages (gratuito) para alojar su JavaDoc y que cualquier persona pueda consultarlo online instantáneamente mediante una URL (ej. <https://tu-usuario.github.io/tu-proyecto/javadoc/>).

3.2. Ejercicio 2.

El ejercicio consistió en crear un paquete propio donde se concentrarán las prácticas y los proyectos realizados en el curso hasta el momento. El paquete se nombró `mx.unam.fi.die.poo.g7`: **mx** corresponde al país, **unam** a la universidad, **fi** a la Facultad de Ingeniería, **die** a la división, **poo** a la asignatura y **g7** al grupo, al llegar al nivel de `g7`, la carpeta se dividió en cuatro áreas de trabajo:

- `src` (Source): Contiene todos los archivos de código fuente (.java).
- `lib` (Libraries): Guarda el código hecho por otras personas que el proyecto necesita para funcionar, como los archivos .jar externos.
- `test` (Pruebas): Se reserva para los códigos que sirven para probar y verificar que los de la carpeta `src` no tengan errores.
- `doc` (Documentación): Guarda los reportes o la documentación generada automáticamente, como Javadoc, para que otras personas entiendan cómo funciona el código.

Una vez creada la estructura, cada archivo tuvo que indicar a qué paquete pertenece. Para esto se escribió al inicio de cada clase la instrucción `package` con la ruta completa del paquete, por ejemplo `mx.unam.fi.die.poo.g7.src.Práctica_1`.

De esta forma la ubicación del archivo dentro de las carpetas coincide con el nombre del paquete que declara.

Finalmente, para garantizar que los archivos generados no se mezclaran con el código fuente y respetar la estructura de áreas de trabajo, se utilizaron comandos de terminal empleando la bandera de destino (`-d`).

Para la compilación, se enviaron los (.class) directamente a la carpeta `test` mediante el comando:

```
javac -d mx/unam/fi/die/poo/g7/test
mx/unam/fi/die/poo/g7/src/practica_5/*.java
```

Para la generación de la documentación técnica automatizada, se ejecutó la herramienta Javadoc indicando que todos los archivos HTML resultantes debían depositarse en la carpeta doc:

```
javadoc -d mx/unam/fi/die/poo/g7/doc  
mx/unam/fi/die/poo/g7/src/practica_5/*.java
```

Posteriormente, la ejecución del programa se realizó invocando el Classpath (-cp) apuntando a la carpeta de pruebas:

```
java -cp mx/unam/fi/die/poo/g7/test  
mx.unam.fi.die.poo.g7.src.practica_5.MainFecha
```

Como prueba, se movió a la practica 1 a la carpeta src dentro del subpaquete Práctica_1, se agregó la declaración del paquete con la ruta completa y se verificó que compila, se ejecuta y da la misma salida que antes.

4. Resultados

4.1. Ejercicio 1.

Se muestra la respuesta obtenida al consultar al LLM.

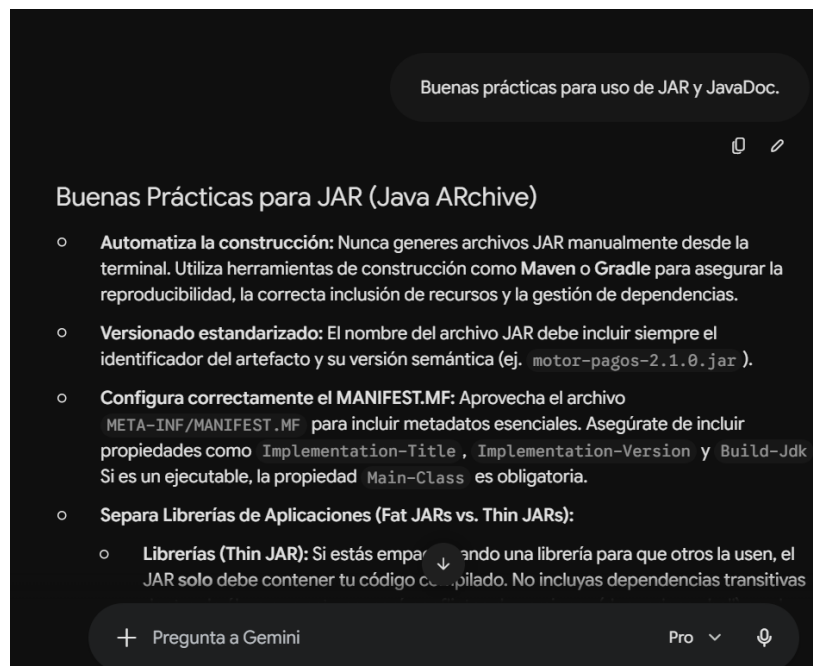


Figura 1: Respuesta de buenas prácticas para uso de JAR y Javadoc.

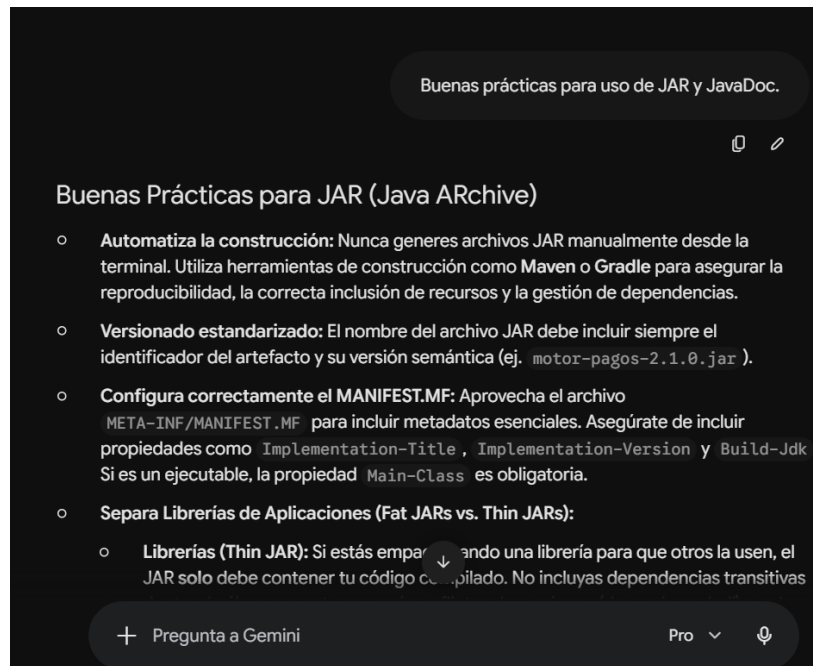


Figura 2: Respuesta de ¿Cuál es la mejor manera de generar un JavaDoc?.

4.2. Resultados del Ejercicio 2.

Se observa la carpeta `src`, la cual contiene de forma ordenada todos los archivos de código fuente (.java) clasificados por práctica y etapa del proyecto.

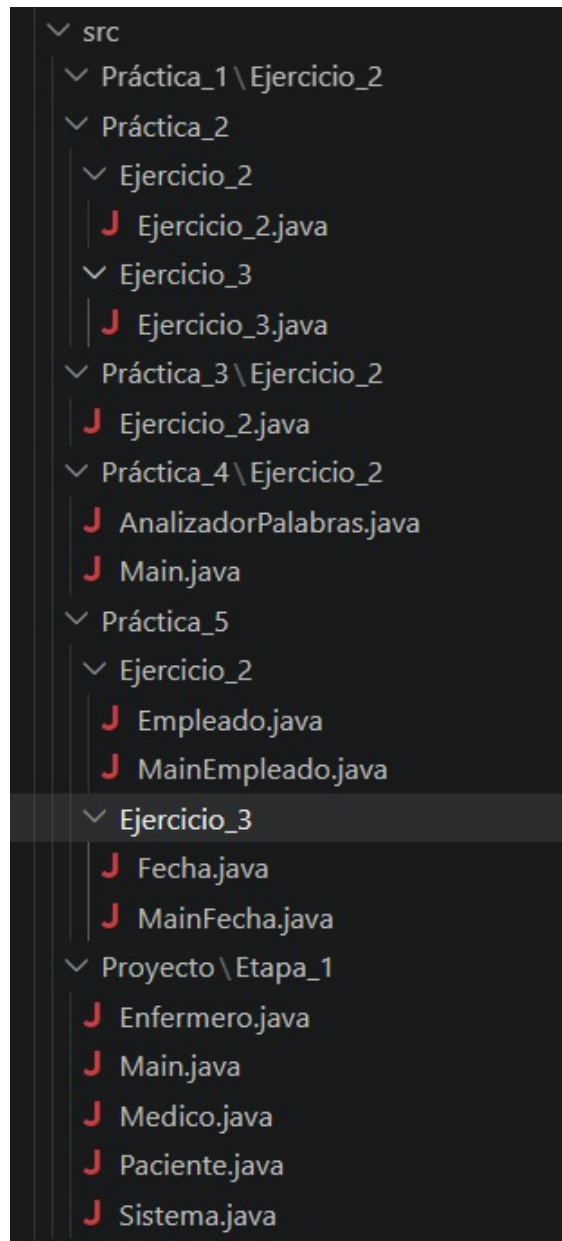


Figura 3: Estructura de archivos fuente (.java) en la carpeta `src`.

Se generaron los archivos compilados (.class), los cuales respetan la misma jerarquía de paquetes definida previamente.

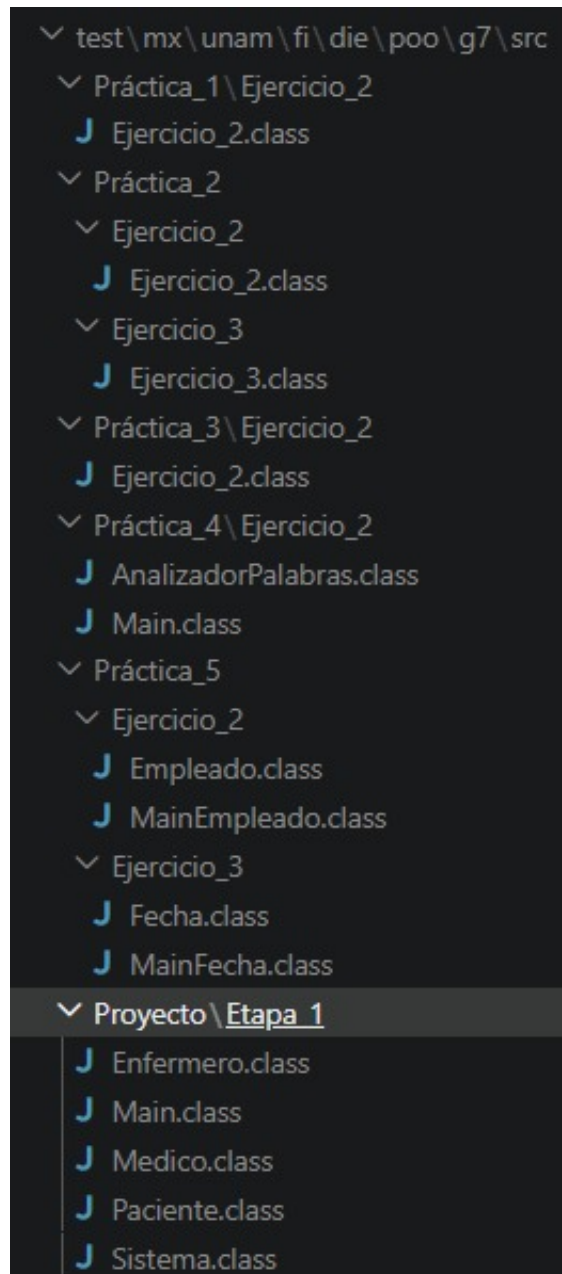


Figura 4: Archivos compilados (.class) organizados en la estructura del proyecto.

Se verificó la generación automatizada de la documentación técnica, la carpeta doc contiene los archivos HTML y recursos generados por la herramienta Javadoc.

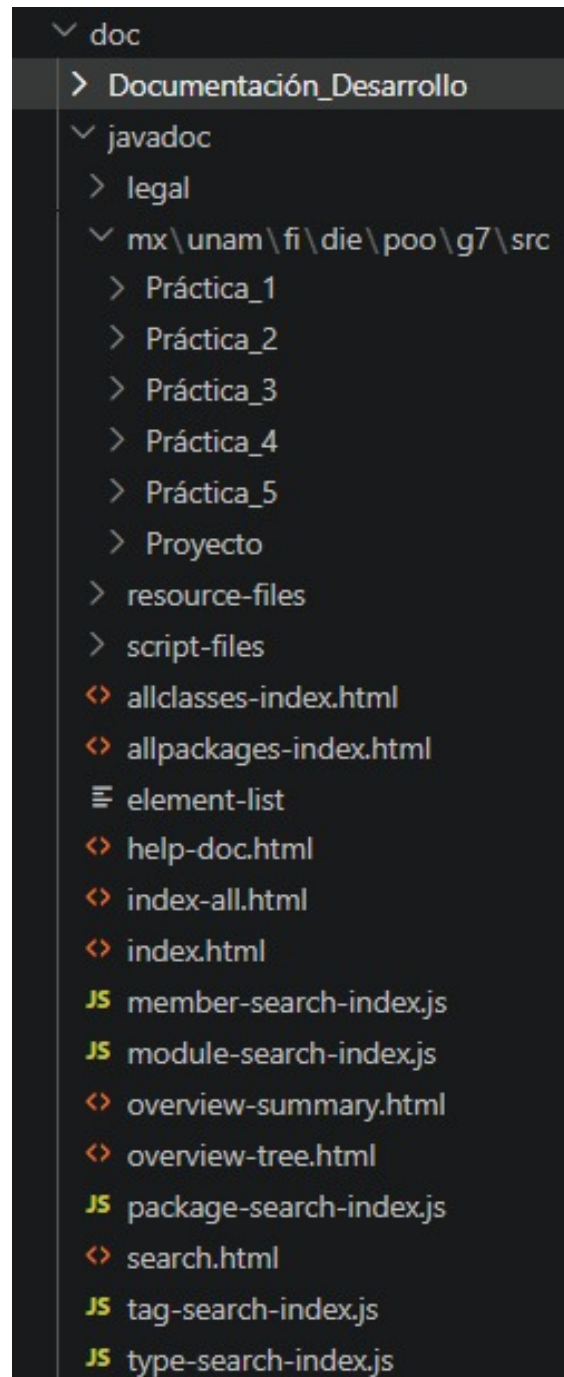


Figura 5: Archivos de documentación autogenerados en la carpeta doc.

Se realizó una prueba de ejecución desde la terminal invocando la ruta completa del paquete, Se ejecutó el programa de la Calculadora Básica de la Práctica 1.

```
$ java -cp mx/unam/fi/die/poo/g7/test mx.unam.fi.die.poo.g7.src.Práctica_1.Ejercicio_2.Ejercicio_2
Bienvenido a la Calculadora Básica
Ingrese el primer número: 6
Ingrese el segundo número:
7

- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 1

El resultado de la suma de 6.0 mas 7.0 = 13.0

- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 6

El resultado de la raíz 7.0 de 6.0 = 1.2917083420907467

- Seleccione una operación -
0. Salir
1. Suma (num1 + num2)
2. Resta (num1 - num2)
3. Multiplicación (num1 * num2)
4. División (num1 / num2)
5. Potencia (num1 ^ num2)
6. Raíz (raíz num2 de num1)
7. Módulo (num1 % num2)
Opción: 0

Se salió exitosamente del programa
```

Figura 6: Ejecución exitosa de un programa invocando la ruta completa del paquete.

5. Conclusiones

El análisis de los resultados demuestra que dividir el proyecto en carpetas específicas (`src`, `lib`, `test`, `doc`) es fundamental para mantener el orden, al aplicar la teoría de darle una "dirección" o nombre único a cada archivo mediante el paquete `mx.unam.fi.die.poo.g7`, se resuelve directamente el problema de la confusión de nombres, garantizando que el trabajo creado no se mezcle ni choque con las herramientas que ya incluye el lenguaje de programación.

Por otro lado, aplicar los conceptos teóricos sobre cómo empaquetar el trabajo (JAR) y cómo generar manuales automáticos (JavaDoc) es lo que permite que otras personas puedan entender y utilizar el programa en el futuro sin necesidad de leer todo el código interno, el uso de esta teoría soluciona el problema inicial de desorganización, logrando que el proyecto deje de ser simplemente un montón de archivos sueltos para convertirse en un trabajo estructurado y fácil de mantener.

Referencias

- [1] API de Java. (s.f.). IBM. Recuperado 8 de octubre de 2026, de <https://www.ibm.com/docs/es/product-master/12.0.0?topic=code-java-api>
- [2] Gálvez, J. A. S. (s.f.). Organización de clases. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3088/mod_resource/content/1/UAPA-Organizacion-Clases/index.html
- [3] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.